

Laravel - najbolje prakse

Autor: Marijan Kopčić **Datum:** 31.08.2026. (Dokument je nastao na temelju blog objave autora Amasa.)

Laravel je moćan framework osmišljen da pojednostavi izradu modernih web aplikacija. Kao i svaki framework, ima najbolje prakse ugrađene u svoju srž. Slijedeći ove smjernice možete pisati čišći kod, smanjiti tehnički dug, poboljšati timsku suradnju i osigurati da vaša baza koda bude u skladu s "Laravel načinom" rada.

U ovom članku istražiti ćemo te ključne Laravel najbolje prakse, od strukturiranja koda do optimizacije operacija nad bazom podataka, kako bi vaši projekti ostali učinkoviti i ugodni za razvoj.

Bilo da ste iskusan Laravel developer ili tek počinjete, ove prakse pomoći će vam da podignete svoje razvojne vještine i isporučite kvalitetne aplikacije.

Krenimo. 📌

1. Debeli modeli, mršavi kontroleri (Fat Models, Skinny Controllers)

Premjestite logiku vezanu uz bazu podataka u Eloquent modele kako biste zadržali čišće kontrolere i ponovno iskoristiv kod.

Loš primjer:

```
public function index()
{
    $clients = Client::verified()
        ->with(['orders' => function ($query) {
            $query->where('created_at', '>', now()->subDays(7));
        }])
        ->get();

    return view('index', compact('clients'));
}
```

Dobar primjer:

```
public function index(Client $client)
{
    return view('index', ['clients' => $client->getVerifiedWithRecentOrders()]);
}

class Client extends Model
{
    public function getVerifiedWithRecentOrders(): Collection
    {
        return $this->verified()
            ->with(['orders' => fn($query) => $query->recent()])
            ->get();
    }

    public function scopeVerified($query)
    {
        return $query->where('is_verified', true);
    }
}

class Order extends Model
{
    public function scopeRecent($query)
    {
        return $query->where('created_at', '>', now()->subDays(7));
    }
}
```

2. Princip jedne odgovornosti (Single Responsibility Principle)

Klasa bi trebala imati samo jednu odgovornost. To znači da bi se klasa trebala fokusirati na jedan dio funkcionalnosti. Kršenje ovog principa čini kod težim za čitanje, testiranje i održavanje jer miješa odgovornosti koje bi trebale biti odvojene.

Pridržavanjem principa jedne odgovornosti stvarate kod koji je lakše razumjeti i refaktorirati. Svaka klasa ili servis ima jasnu svrhu, čime cijeli sustav postaje modularniji i fleksibilniji.

Loš primjer:

```

public function update(Request $request): string
{
    $validated = $request->validate([
        'name' => 'required|max:255',
        'tasks' => 'required|array:due_date,status'
    ]);

    foreach ($request->tasks as $task) {
        $formattedDate = $this->carbon->parse($task['due_date'])->toDateTimeString();
        $this->logger->info('Task updated: ' . $formattedDate . ' - ' . $task['status']);
    }

    $this->project->updateTasks($request->validated());

    return redirect()->route('projects.index');
}

```

Dobar primjer:

```

public function update(UpdateProjectRequest $request): string
{
    $this->taskLogger->logTasks($request->tasks);
    $this->projectService->updateTasks($request->validated());

    return redirect()->route('projects.index');
}

class TaskLogger
{
    public function logTasks(array $tasks): void
    {
        // Logika za logiranje zadataka
    }
}

class ProjectService
{
    public function updateTasks(array $data): void
    {
        // Logika za ažuriranje zadataka projekta
    }
}

```

3. Metode trebaju raditi samo jednu stvar

Funkcija bi trebala imati jednu svrhu i izvršavati je dobro. Kada metoda radi više od jedne stvari, postaje ju teže razumjeti, testirati i održavati. Razdvajanje odgovornosti u manje, fokusirane metode čini kod čitljivijim i lakšim za debugiranje.

Loš primjer:

```
public function getFullNameAttribute(): string
{
    if (auth()->user() && auth()->user()->hasRole('admin') && auth()->user()->isVerified()) {
        return 'Admin ' . $this->first_name . ' ' . $this->last_name;
    } else {
        return $this->first_name[0] . ' ' . $this->last_name;
    }
}
```

Dobar primjer:

```
public function getFullNameAttribute(): string
{
    return $this->isVerifiedAdmin() ? $this->formatFullName() : $this->formatShortName();
}

private function isVerifiedAdmin(): bool
{
    $user = auth()->user();
    return $user && $user->hasRole('admin') && $user->isVerified();
}

private function formatFullName(): string
{
    return 'Admin ' . $this->first_name . ' ' . $this->last_name;
}

private function formatShortName(): string
{
    return strtoupper($this->first_name[0]) . ' ' . ucfirst($this->last_name);
}
```

4. Držite poslovnu logiku u servisnim klasama

Kontroleri bi trebali samo obrađivati HTTP zahtjeve i odgovore, a složenu logiku delegirati servisnim klasama. To čini kod čistim, ponovno iskoristivim i lakšim za testiranje.

Loš primjer:

```
public function store(Request $request)
{
    if ($request->hasFile('image')) {
        $image = $request->file('image');
        $image->storeAs('temp', $image->getClientOriginalName(), 'public');
    }

    // Ostala nepovezana logika...
}
```

Dobar primjer:

```
public function store(Request $request, ArticleService $articleService)
{
    $articleService->uploadImage($request->file('image'));

    // Ostala nepovezana logika...
}

class ArticleService
{
    public function uploadImage(?UploadedFile $image): void
    {
        if ($image) {
            $image->storeAs('uploads/temp', uniqid() . '_' . $image->getClientOriginalName(),
'public');
        }
    }
}
```

5. Izbjegavajte poslovnu logiku u rutama

Rute bi trebale samo obrađivati HTTP zahtjeve, a ne poslovnu logiku. Time se kod održava čistim i lakšim za održavanje.

Loš primjer:

```
// Poslovna logika u ruti
Route::post('/article', function (Request $request) {
    $article = new Article;
    $article->title = $request->title;
    $article->content = $request->content;
    $article->save();
});
```

Dobar primjer:

```
// Ruta delegira logiku kontroleru
Route::post('/article', [ArticleController::class, 'store']);

// U ArticleController
public function store(Request $request)
{
    // logika za kreiranje članka
}
```

6. Koristite relacije za čišći kod

Koristite Eloquent relacije kako biste pojednostavili i pojasnili kako povezani modeli međusobno komuniciraju. Time se izbjegavaju ponavljajuća pridruživanja vrijednosti, a kod postaje lakši za održavanje i manje podložan greškama.

Loš primjer:

```
$article = new Article;
$article->title = $request->input('title');
$article->content = $request->input('content');
$article->verified = $request->boolean('verified');
$article->category_id = $category->id;
$article->save();
```

Dobar primjer:

```
$category->articles()->create($request->safe()->only(['title', 'content', 'verified']));
```

7. Koristite transakcije baze za atomarne poslovne operacije

Transakcije osiguravaju da sve operacije nad bazom uspiju ili ne uspiju kao cjelina, čime se održava integritet podataka.

Loš primjer:

```
public function placeOrder(Request $request)
{
    $order = new Order;
    $order->user_id = $request->user_id;
    $order->save();

    $payment = new Payment;
    $payment->order_id = $order->id;
    $payment->save();
}
```

Dobar primjer:

```
use DB;

public function placeOrder(Request $request)
{
    DB::beginTransaction();

    try {
        $order = Order::create($request->validated());
        $payment = Payment::create(['order_id' => $order->id]);

        DB::commit();
    } catch (\Exception $e) {
        DB::rollBack();
        throw $e;
    }
}
```

8. Izbjegavajte upite u Bladeu: koristite eager loading

Izvršavanje upita unutar Blade predložaka dovodi do neučinkovitih poziva prema bazi, osobito unutar petlji. Eager loading dohvaća povezane podatke u jednom upitu, čime se poboljšavaju performanse i izbjegava N + 1 problem.

Loš primjer:

```
@foreach (User::all() as $user)
    {{ $user->profile->name }}
@endforeach
```

Ako imate 100 korisnika, ovo pokreće 101 upit: jedan za korisnike i jedan za profil svakog korisnika.

Dobar primjer:

```
// u servisnoj klasi ili modelu koji se vraća kontroleru, a koji to prosljeđuje Blade datoteci
$users = User::with('profile')->get();
```

```
{{-- u Blade datoteci --}}
@foreach ($users as $user)
    {{ $user->profile->name }}
@endforeach
```

Ovo pokreće samo 2 upita: jedan za korisnike i jedan za njihove profile.

9. Obradujte podatke u dijelovima (chunk) radi performansi

Za zadatke koji uključuju velike skupove podataka, obrada u dijelovima smanjuje potrošnju memorije i poboljšava performanse ograničavanjem količine podataka koja se istovremeno drži u memoriji.

Loš primjer:

```
$users = User::all();

foreach ($users as $user) {
    // Obrada svakog korisnika
}
```

Dobar primjer:

```
User::chunk(500, function ($users) {
    foreach ($users as $user) {
        // Obrada svakog korisnika
    }
});
```

10. Koristite konstante umjesto tvrdo kodiranih vrijednosti

Korištenje konstanti pomoći će vam da pronađete mjesta na kojima se ta vrijednost koristi, u slučaju da je želite promijeniti, te će vam pomoći kod refaktoriranja i debugiranja.

Loš primjer:

```
public function isAdmin(User $user): bool
{
    return $user->type === 'admin';
}
```

Dobar primjer:

```
public function isAdmin(User $user)
{
    return $user->type === UserType::ADMIN;
}
```

11. Prevodite stringove

Zahvalit ćete si u budućnosti, kako vaša aplikacija bude rasla, ako ste od početka razmišljali o prijevodu stringova. Sve što trebate je proslijediti stringove kroz funkciju `__()`.

Loš primjer:


```
return back()->with('message', 'Your article has been added!');
```

Dobar primjer:

```
return back()->with('message', __('Your article has been added!')); // primijetite poziv na __()
```

12. Injektirajte ovisnosti (Dependency Injection)

Stvaranje instanci pomoću `new` čvrsto povezuje vaše klase i otežava njihovo testiranje ili mijenjanje. Korištenje IoC kontejnera omogućuje jednostavniju injekciju ovisnosti i bolju testabilnost.

Loš primjer:

```
public function store(Request $request)
{
    $user = new User;
    $user->create($request->validated());
}
```

Dobar primjer:

```
public function __construct(protected UserService $userService) {}

public function store(Request $request)
{
    $this->userService->create($request->validated());
}
```

13. Izbjegavajte izravno korištenje `.env` u kodu

Izravno dohvaćanje podataka iz `.env` datoteke kroz cijelu aplikaciju može otežati održavanje i testiranje koda. Umjesto toga, spremite vrijednosti u konfiguracijske datoteke i dohvaćajte ih pomoću `config()`.

Loš primjer:

```
$apiKey = env('API_KEY');
```

Dobar primjer:

```
// config/services.php
'api_key' => env('API_KEY'),

// Dohvaćanje vrijednosti
$apiKey = config('services.api_key');
```

14. Spremajte datume kao objekte, ne kao stringove

Spremanje datuma kao stringova može dovesti do nekonzistentnih formata i grešaka pri parsiranju. Bolje ih je spremati kao Carbon instance, koje pružaju robusno rukovanje datumima. Koristite `accessore` i `mutatore` kako biste datume formatirali samo kada je to potrebno u sloju prikaza.

Loš primjer:

```
{{ Carbon::createFromFormat('Y-d-m H-i', $object->ordered_at)->toDateString() }}
{{ Carbon::createFromFormat('Y-d-m H-i', $object->ordered_at)->format('m-d') }}
```

Dobar primjer:

```
// U modelu
protected $casts = [
    'ordered_at' => 'datetime',
];
```

```
{{!-- U Blade prikazu --}}
{{ $object->ordered_at->toDateString() }}
{{ $object->ordered_at->format('m-d') }}
```

15. Držite dokumentaciju koda minimalnom i smislenom

Pretjerana dokumentacija često zatrpava kod i otežava njegovo održavanje. Umjesto toga, oslonite se na jasna, opisna imena varijabli, funkcija i klasa. Komentare koristite samo kada je apsolutno nužno objasniti složenu logiku.

Loš primjer:

```
/**
 * The function checks if the given string has any white spaces
 *
 * @param string $string String received from frontend which might contain
 *                        space characters. Returns True if the string
 *                        is valid.
 *
 * @return bool
 *
 * @license GPL
 */

public function checkString($string)
{
}
```

Dobar primjer:

```
public function hasWhiteSpaces(string $string): bool
{
}
```

16. Uskladite se s timom oko standarda kodiranja

Konzistentan kod poboljšava čitljivost i održivost te olakšava suradnju.

Možete koristiti **Laravel Pint** za automatsko formatiranje i provođenje standarda kodiranja. Integrira se u vaš razvojni proces, pa ga možete pokrenuti prije svakog commita koristeći **Git Hooks**.

```
composer require --dev laravel/pint
vendor/bin/pint
```

17. Testirajte, testirajte i testirajte

I na kraju, jedna od najvažnijih stvari koje možete učiniti kako biste osigurali pouzdanost, održivost i skalabilnost svojeg koda jest pisanje automatiziranih testova.

Ne trebate 100 % pokrivenost funkcionalnosti (tvrdio bih da je to kontraproduktivno), ali barem trebate osigurati da su sve vaše GET rute pokrivene — a što više možete dodati povrh toga, to bolje.

Evo zašto:

1. **Rano otkrivanje bugova:** Testovi pomažu identificirati probleme prije nego što dođu u produkciju. Hvatanje bugova tijekom razvoja daleko je jeftinije i lakše nego njihovo popravljavanje nakon deploya.
2. **Povjerenje u kod:** Uz odgovarajuću pokrivenost testovima možete s pouzdanjem mijenjati bazu koda. Testovi osiguravaju da nove promjene ne razbiju postojeću funkcionalnost.
3. **Dokumentacija:** Dobro napisani testovi djeluju kao živa dokumentacija vašeg koda. Opisuju kako se sustav treba ponašati i mogu se koristiti za razumijevanje namjere koda.
4. **Sigurno refaktoriranje:** Kod refaktoriranja ili poboljšavanja postojećeg koda, testovi pružaju sigurnosnu mrežu koja osigurava da se tijekom promjena ne izgubi nikakva funkcionalnost.
5. **Bolji dizajn:** Pisanje testova često potiče bolji dizajn softvera. Da biste pisali testabilan kod, obično završite s manjim, fokusiranim metodama i klasama koje je lakše održavati.
6. **Suradnja:** Testovi olakšavaju timovima zajednički rad na velikim bazama koda. Definiraju jasna očekivanja ponašanja, čime se smanjuju nesporazumi i poboljšava suradnja.
7. **Kontinuirana integracija:** Testovi su ključni za implementaciju kontinuirane integracije i isporuke (CI/CD). Automatizirani testovi mogu se pokretati pri svakom pushu koda, osiguravajući da se deploya samo stabilan kod.
8. **Dugoročno održavanje:** U velikim projektima testovi pomažu održati stabilnost tijekom vremena, osobito kada se tim mijenja. Novi developeri mogu se osloniti na testove kako bi razumjeli ponašanje baze koda i osigurali da buduće promjene ništa ne razbiju.

Dodatne prakse

Sljedeće stavke nadopunjuju gornji popis, a preuzete su iz dva dodatna izvora: [alexeymezenin/laravel-best-practices](#) i [Strapi - Laravel Best Practices](#).

18. Validaciju premjesti u Form Request klase

Validacijska pravila ne drži u kontroleru — premjesti ih u zasebnu Request klasu. Kontroler tako ostaje čist, a pravila su ponovno iskoristiva i lakše se testiraju.

Loš primjer:

```
public function store(Request $request)
{
    $request->validate([
        'title' => 'required|unique:posts|max:255',
        'body' => 'required',
        'publish_at' => 'nullable|date',
    ]);

    // ...
}
```

Dobar primjer:

```
public function store(PostRequest $request)
{
    // ...
}

class PostRequest extends FormRequest
{
    public function rules(): array
    {
        return [
            'title' => 'required|unique:posts|max:255',
            'body' => 'required',
            'publish_at' => 'nullable|date',
        ];
    }
}
```

19. Ne ponavljaj se (DRY)

Ponovno iskoristi kod gdje god možeš — kroz Eloquent scopeove, zajedničke metode i ponovno korištenje Blade predložaka.

Loš primjer:

```
public function getActive()
{
    return $this->where('verified', 1)->whereNotNull('deleted_at')->get();
}

public function getArticles()
{
    return $this->whereHas('user', function ($q) {
        $q->where('verified', 1)->whereNotNull('deleted_at');
    }->get();
}
```

Dobar primjer:

```
public function scopeActive($q)
{
    return $q->where('verified', true)->whereNotNull('deleted_at');
}

public function getActive()
{
    return $this->active()->get();
}

public function getArticles()
{
    return $this->whereHas('user', fn($q) => $q->active())->get();
}
```

20. Preferiraj Eloquent umjesto Query Buildera i sirovog SQL-a; preferiraj kolekcije umjesto polja

Eloquent omogućuje pisanje čitljivog i održivog koda te nudi ugrađene alate poput soft deletea, eventova, scopeova i sl. Kolekcije imaju moćne metode koje sirova polja nemaju.

Loš primjer:

```

SELECT *
FROM `articles`
WHERE EXISTS (SELECT *
              FROM `users`
              WHERE `articles`.`user_id` = `users`.`id`
              AND EXISTS (SELECT *
                          FROM `profiles`
                          WHERE `profiles`.`user_id` = `users`.`id`))
              AND `users`.`deleted_at` IS NULL)
AND `verified` = '1'
AND `active` = '1'
ORDER BY `created_at` DESC

```

Dobar primjer:

```
Article::has('user.profile')->verified()->latest()->get();
```

21. Mass assignment

Koristi Eloquentovo mass assignment s validiranim podacima umjesto ručnog pridruživanja svakog svojstva.

Loš primjer:

```

$article = new Article;
$article->title = $request->title;
$article->content = $request->content;
$article->verified = $request->verified;
// Pridruživanje kategorije
$article->category_id = $category->id;
$article->save();

```

Dobar primjer:

```
$category->article()->create($request->validated());
```

22. Ne stavljaš JS i CSS u Blade predloške niti HTML u PHP klase

Frontend resurse drži odvojeno od predložaka, a HTML izvan PHP klase.

Loš primjer:

```
let article = `{{ json_encode($article) }}`;
```

Bolji primjer:

```
<input id="article" type="hidden" value="@json($article)">

{{-- ili --}}

<button class="js-fav-article" data-article="@json($article)">{{ $article->name }}</button>
```

U JavaScript datoteci:

```
let article = $('#article').val();
```

Najbolji način je korištenje specijaliziranog PHP-to-JS paketa za prijenos podataka.

23. Koristi konfiguracijske i jezične datoteke te konstante umjesto teksta u kodu

Tekstove, postavke i konstante ne piši tvrdo u kodu — smjesti ih u konfiguracijske i jezične datoteke, odnosno u konstante klase.

Loš primjer:

```
public function isNormal(): bool
{
    return $article->type === 'normal';
}

return back()->with('message', 'Your article has been added!');
```

Dobar primjer:

```
public function isNormal()
{
    return $article->type === Article::TYPE_NORMAL;
}

return back()->with('message', __('app.article_added'));
```

24. Koristi standardne Laravel alate prihvaćene u zajednici

Prednost daj ugrađenim Laravel značajkama i paketima koje zajednica koristi. Svaki drugi paket znači dodatnu stvar koju ti i tvoj tim morate učiti i održavati, a i dobivanje pomoći od zajednice je teže.

Zadatak	Standardni alati	Alati trećih strana
Autorizacija	Policies	Entrust, Sentinel
Kompajliranje resursa	Laravel Mix, Vite	Grunt, Gulp
Razvojno okruženje	Laravel Sail, Homestead	Docker
Deployment	Laravel Forge	Deployer
Unit testiranje	PHPUnit, Mockery	Phpspec, Pest
Browser testiranje	Laravel Dusk	Codeception
Baza podataka	Eloquent	SQL, Doctrine
Predlošci	Blade	Twig
Rad s podacima	Laravel kolekcije	Polja (arrays)
Validacija formi	Request klase	Paketi trećih strana
Autentikacija	Ugrađena	Paketi trećih strana
API autentikacija	Laravel Passport, Laravel Sanctum	JWT, OAuth paketi
Izrada API-ja	Ugrađeno	Dingo API
Struktura baze	Migracije	Direktan rad nad bazom
Lokalizacija	Ugrađena	Paketi trećih strana
Realtime UI	Laravel Echo, Pusher	Paketi trećih strana
Testni podaci	Seeder klase, Model Factories, Faker	Ručno kreiranje
Raspoređivanje zadataka	Laravel Task Scheduler	Skripte, paketi trećih strana

Napomena: izvorna tablica svrstava Pest među alate trećih strana. Interna pravila ovog projekta (vidi sekciju "Upute za AI") ipak nalažu Pest kao obavezan alat za testiranje — ta pravila imaju prednost.

25. Slijedi Laravel konvencije imenovanja

Što	Kako	Dobro	Loše
Kontroler	jednina	<code>ArticleController</code>	<code>ArticlesController</code>
Ruta	množina	<code>articles/1</code>	<code>article/1</code>
Naziv rute	snake_case s točkom	<code>users.show_active</code>	<code>users.show-active</code>
Model	jednina	<code>User</code>	<code>Users</code>
hasOne / belongsTo relacija	jednina	<code>articleComment</code>	<code>articleComments</code>
Sve ostale relacije	množina	<code>articleComments</code>	<code>articleComment</code>
Tablica	množina	<code>article_comments</code>	<code>article_comment</code>
Pivot tablica	nazivi modela u jednini, abecedno	<code>article_user</code>	<code>user_article</code>
Stupac tablice	snake_case bez naziva modela	<code>meta_title</code>	<code>MetaTitle</code>
Svojstvo modela	snake_case	<code>\$model->created_at</code>	<code>\$model->createdAt</code>
Strani ključ	naziv modela u jednini sa sufixom <code>_id</code>	<code>article_id</code>	<code>ArticleId</code>
Primarni ključ	—	<code>id</code>	<code>custom_id</code>
Migracija	—	<code>2017_01_01_000000_create_articles_table</code>	<code>2017_01_01_000000_articles</code>
Metoda	camelCase	<code>getAll</code>	<code>get_all</code>
Metoda u resource kontroleru	prema tablici resource metoda	<code>store</code>	<code>saveArticle</code>
Metoda u testnoj klasi	camelCase	<code>testGuestCannotSeeArticle</code>	<code>test_guest_cannot_see_article</code>
Varijabla	camelCase	<code>\$articlesWithAuthor</code>	<code>\$articles_with_author</code>
Kolekcija	opisno, množina	<code>\$activeUsers = User::active()->get()</code>	<code>\$active</code>
Objekt	opisno, jednina	<code>\$activeUser = User::active()->first()</code>	<code>\$users</code>
Ključ u config/jezičnim datotekama	snake_case	<code>articles_enabled</code>	<code>ArticlesEnabled</code>

Što	Kako	Dobro	Loše
View	kebab-case	show-filtered.blade.php	showFiltered.blade.php
Config datoteka	snake_case	google_calendar.php	googleCalendar.php
Contract (sučelje)	pridjev ili imenica	Authenticatable , AuthenticationInterface	—
Trait	pridjev	Notifiable	NotificationTrait
Enum	jednina	UserType	UserTypes
FormRequest	jednina	UpdateUserRequest	UpdateUserFormRequest
Seeder	jednina	UserSeeder	UsersSeeder

26. Konvencija ispred konfiguracije

Ako slijediš Laravel konvencije, ne moraš pisati dodatnu konfiguraciju.

Loš primjer:

```
// Tablica nije imenovana po konvenciji
class Article extends Model
{
    protected $table = 'article';
    protected $primaryKey = 'article_id';

    public function comments()
    {
        return $this->hasMany(Comment::class, 'article_comment_id', 'article_id');
    }
}
```

Dobar primjer:

```
class Article extends Model
{
    public function comments()
    {
        return $this->hasMany(Comment::class);
    }
}
```

27. Koristi kraću i čitljiviju sintaksu gdje je moguće

Uobičajena sintaksa	Kraća / čitljivija
<code>Session::get('cart')</code>	<code>session('cart')</code>
<code>\$request->session()->get('cart')</code>	<code>session('cart')</code>
<code>Session::put('cart', \$data)</code>	<code>session(['cart' => \$data])</code>
<code>\$request->input('name')</code>	<code>\$request->name</code>
<code>return Redirect::back()</code>	<code>return back()</code>
<code>is_null(\$object->relation) ? null : \$object->relation->id</code>	<code>optional(\$object->relation)->id</code>
<code>return view('index')->with('title', \$title)->with('client', \$client)</code>	<code>return view('index', compact('title', 'client'))</code>
<code>\$request->has('value') ? \$request->value : 'default'</code>	<code>\$request->get('value', 'default')</code>
<code>Carbon::now()</code>	<code>now()</code>
<code>->where('column', '=', 1)</code>	<code>->where('column', 1)</code>
<code>->orderBy('created_at', 'desc')</code>	<code>->latest()</code>
<code>->select('id', 'name')->get()</code>	<code>->get(['id', 'name'])</code>
<code>->first()->name</code>	<code>->value('name')</code>

28. Ostale dobre prakse

- Izbjegavaj obrasce i alate koji su strani Laravelu i sličnim frameworkovima (npr. RoR, Django).
- Nikad ne stavljalj nikakvu logiku u datoteke ruta.
- Svedi korištenje čistog PHP-a u Blade predlošcima na minimum.
- Za testiranje koristi in-memory bazu podataka.
- Nemoj prepisivati (override) standardne značajke frameworka kako bi izbjegao probleme pri nadogradnji verzije frameworka.
- Koristi modernu PHP sintaksu gdje je moguće, ali ne zaboravi na čitljivost.
- Izbjegavaj View Composere i slične alate osim ako stvarno znaš što radiš.

29. Slijedi PSR standarde kodiranja

PSR-2 je službeno proglašen zastarjelim 2019. godine. Danas se koriste **PSR-12** i **PER Coding Style**, koji pokrivaju modernu PHP sintaksu (atributi, enumi, promoted properties, tipovi). Ovi standardi definiraju uvlačenje, položaj vitičastih zagrada, deklaracije tipova i organizaciju `use` naredbi. Provedbu automatiziraj alatom (Laravel Pint, vidi stavku 16) i pokretanjem u CI-ju, umjesto da se oslanjaš na ručne code review komentare.

30. Ostani na aktualnoj verziji Laravela

Svaka Laravel verzija ima ograničen prozor u kojem prima sigurnosne zakrpe (kod izdanja s dugoročnom podrškom to je otprilike dvije godine za sigurnosne popravke). Zaostajanje znači da radiš na kodu koji više ne dobiva sigurnosne ispravke, a nadogradnja preko više glavnih verzija odjednom uvijek je bolnija od redovitih, malih koraka.

Praktično: prati službene napomene o nadogradnji, redovito pokreći `composer update` uz zaključan constraint, i drži testove zelenima kako bi nadogradnja bila sigurna. Provjeri i minimalnu PHP verziju koju tražena Laravel verzija zahtijeva.

31. Iznudi HTTPS u produkciji

Sav promet u produkciji mora biti šifriran. Umjesto oslanjanja na to da će svaki link biti ispravno napisan, iznudi HTTPS na razini generiranja URL-ova.

```
// app/Providers/AppServiceProvider.php
public function boot(): void
{
    if ($this->app->environment('production')) {
        URL::forceHttps();
    }
}
```

32. Autentikacija i autorizacija temeljena na ulogama

Ne provjeravaj dozvole ad hoc kroz kod. Definiraj granularna prava pomoću **Gateova** i **Policyja**, pa ih dosljedno primjenjuj u kontrolerima, rutama i prikazima.

```
// Policy
public function update(User $user, Post $post): bool
{
    return $user->id === $post->user_id;
}

// Kontroler
$this->authorize('update', $post);
```

```
@can('update', $post)
    {{-- gumb za uređivanje --}}
@endcan
```

33. Rate limiting i sigurnosna zaglavlja

Imenovani limiteri štite od brute force napada na prijavu i od zloupotrebe API-ja, a Content Security Policy smanjuje površinu za XSS.

```
// app/Providers/AppServiceProvider.php
RateLimiter::for('login', function (Request $request) {
    return Limit::perMinute(5)->by($request->ip());
});
```

```
Route::post('/login', LoginController::class)->middleware('throttle:login');
```

Sigurnosna zaglavlja (CSP, X-Content-Type-Options, Referrer-Policy, HSTS) dodaj kroz middleware ili na razini web poslužitelja.

34. Cachiraj konfiguraciju, rute, viewove i rezultate upita

Cachiranje uklanja ponavljajući posao koji se inače izvršava pri svakom zahtjevu.

```
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan event:cache
```

Za skupe upite koristi cache s ključem i razumnim trajanjem:

```
$stats = Cache::remember('dashboard.stats', now()->addMinutes(15), function () {
    return Order::selectRaw('status, count(*) as total')->groupBy('status')->get();
});
```

Oprez: nakon `config:cache` funkcija `env()` izvan konfiguracijskih datoteka vraća `null` — još jedan razlog za pravilo iz stavke 13.

35. Dodatno optimiziraj upite prema bazi

Osim eager loadinga (stavka 8):

- **Spriječi lazy loading u razvoju** kako bi $N + 1$ problemi odmah pukli:

```
// app/Providers/AppServiceProvider.php
Model::preventLazyLoading(! $this->app->isProduction());
```

- **Dohvaćaj samo potrebne stupce** umjesto `SELECT *`:

```
User::get(['id', 'name', 'email']);
```

- **Dodaj indekse** na stupce po kojima filtriraš, sortiraš i spajaš tablice:

```
$table->index(['status', 'created_at']);
```

36. Statičke resurse posluži preko CDN-a

Kompajlirane resurse (JS, CSS, slike) posluži globalno preko CDN-a postavljanjem varijable okoline:

```
ASSET_URL=https://cdn.example.com
```

Nakon toga `asset()` i Vite helperi automatski generiraju CDN URL-ove.

37. Optimiziraj slike i ostale resurse

- Za manipulaciju slikama u runtimeu (resize, crop, konverzija formata) koristi **Intervention Image**.
- Za kompresiju na razini datoteka koristi alate poput **Spatie Image Optimizer** paketa.
- Generiraj responzivne varijante i moderne formate (WebP/AVIF) umjesto posluživanja originala pune veličine.

38. Unit i feature testovi

Stavka 17 pokriva *zašto* testirati; ovdje je *što* pisati:

- **Unit testovi** — izolirano testiraju pojedinačnu logiku (servisi, value objekti, helperi), bez baze i HTTP sloja.
- **Feature testovi** — pokrivaju cijeli ciklus zahtjev-odgovor: rutu, middleware, validaciju, bazu i odgovor.

```
it('creates a post', function () {
    $user = User::factory()->create();

    $this->actingAs($user)
        ->post(route('posts.store'), ['title' => 'Test', 'body' => 'Sadržaj'])
        ->assertRedirect();

    expect(Post::where('title', 'Test')->exists())->toBeTrue();
});
```

39. Automatiziraj CI/CD

Testovi koje nitko ne pokreće ne vrijede ništa. Postavi workflow (npr. GitHub Actions) koji na svaki pull request pokreće testove, Pint i statičku analizu, i koji blokira merge ako nešto pukne.

```
# .github/workflows/tests.yml (skraćeno)
on: [pull_request]
jobs:
  tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with:
          php-version: '8.3'
      - run: composer install --no-interaction --prefer-dist
      - run: vendor/bin/pint --test
      - run: php artisan test
```

40. Migracija za svaku promjenu sheme

Nikad ne mijenjaj shemu baze ručno. Svaka promjena ide kroz migraciju koja je u verzioniranju, koja se može pregledati u code reviewu i koja ima ispravno napisanu `down()` metodu za sigurno vraćanje unatrag.

```
public function up(): void
{
    Schema::table('posts', function (Blueprint $table) {
        $table->string('slug')->nullable()->after('title');
    });
}

public function down(): void
{
    Schema::table('posts', function (Blueprint $table) {
        $table->dropColumn('slug');
    });
}
```

41. Automatiziraj backup

Postavi dnevni backup aplikacije i baze (npr. paketom `spatie/laravel-backup`) sa spremanjem izvan produkcijskog poslužitelja — S3 ili sličan objektni storage. Backup koji nikad nije testiran nije backup: povremeno provjeri da se restore stvarno izvršava.

```
// routes/console.php
Schedule::command('backup:clean')->daily()->at('01:00');
Schedule::command('backup:run')->daily()->at('01:30');
```

42. Laravel uz headless CMS (opcionalno)

Kad sadržajem upravljaju urednici, a ne developeri, razmotri odvajanje upravljanja sadržajem od aplikacijske logike pomoću headless CMS-a (npr. Strapi). CMS tada pokriva uredničke tokove i objavljivanje, dok Laravel ostaje zadužen za autentikaciju, poslovna pravila i prikaz.

Integracija ide kroz Laravelov HTTP klijent uz cachiranje, kako se isti sadržaj ne bi dohvaćao pri svakom zahtjevu:

```
$articles = Cache::remember('cms.articles', now()->addMinutes(10), function () {
    return Http::withToken(config('services.cms.token'))
        ->get(config('services.cms.url') . '/api/articles')
        ->json('data');
});
```

(Ova je stavka preuzeta iz Strapijevog članka i po prirodi je arhitektonska preporuka, a ne univerzalno Laravel pravilo.)

Upute za AI (CLAUDE / LLM)

Zadnje ažuriranje: 23. siječnja 2026.

Ovaj dio je namijenjen AI asistentima (Claude, odnosno bilo koji LLM) koji rade na ovom projektu. Pravila se primjenjuju bez iznimke.

Opće upute za kod

- U PHP-u nemoj generirati komentare iznad metoda ili blokova koda ako su očiti. Nemoj dodavati docblock komentare pri definiranju varijabli, osim ako to nije izričito traženo, npr. `/** @var \App\Models\User $currentUser */`. Komentare piši samo za ono što zahtijeva dodatno objašnjenje zašto je kod tako napisan.
- Za nove značajke **MORAŠ** generirati automatizirane Pest testove.
- Za dokumentaciju biblioteka: ako neka biblioteka nije dostupna u Laravel Boost alatu `search-docs`, uvijek koristi Context7. Automatski koristi Context7 MCP alate za dohvat library ID-a i dokumentacije, bez da to moram posebno tražiti.

PHP upute

- U PHP-u koristi operator `match` umjesto `switch` kad god je to moguće.

- Enume uvijek generiraj u mapi `app/Enums`, a ne u glavnoj mapi `app/`, osim ako nije drugačije navedeno.
 - Ako vrijednosti stupca dolaze iz enuma, u migraciji uvijek koristi vrijednost enuma kao default. Taj stupac u modelu uvijek castaj u tip enuma.
 - Nemoj stvarati privremene varijable poput `$currentUser = auth()->user()` ako se ta varijabla koristi samo jednom.
 - Uvijek koristi Enum umjesto tvrdo kodiranih string vrijednosti, gdje je to moguće i gdje Enum klasa postoji. Na primjer, u Blade datotekama i u testovima pri kreiranju podataka — ako je polje castano u Enum, koristi taj Enum umjesto tvrdo kodirane vrijednosti.
-

Laravel upute

- **Korištenje servisa u kontrolerima:** ako se Service klasa koristi samo u JEDNOJ metodi kontrolera, injektiraj je direktno u tu metodu preko type-hintanja. Ako se Service klasa koristi u VIŠE metoda kontrolera, inicijaliziraj je u konstruktoru.
 - **Eloquent Observere** registriraj u Eloquent modelima pomoću PHP atributa, a ne u `AppServiceProvider`. Primjer: `#[ObservedBy([UserObserver::class])]` uz `use Illuminate\Database\Eloquent\Attributes\ObservedBy;` na vrhu.
 - Teži "slim" kontrolerima i veće dijelove logike stavlja u Service klase.
 - Koristi Laravel helpere umjesto klasa iz `use` sekcije. Primjeri: koristi `auth()->id()` umjesto `Auth::id()` uz dodavanje `Auth` u `use` sekciju. Ostali primjeri: koristi `redirect()->route()` umjesto `Redirect::route()`, ili `str()->slug()` umjesto `Str::slug()`.
 - Nemoj koristiti `whereKey()` ni `whereKeyNot()`, koristi konkretna polja poput `id`. Primjer: umjesto `>whereKeyNot($currentUser->getKey())`, koristi `->where('id', '!=', $currentUser->id)`.
 - Nemoj dodavati `::query()` pri izvršavanju Eloquent `create()` naredbi. Primjer: umjesto `User::query()->create()`, koristi `User::create()`.
 - U Livewire projektima nemoj koristiti Livewire Volt. Isključivo Livewire klasne komponente.
 - Kad u migraciji dodaješ stupce, ažuriraj `$fillable` polje modela tako da uključuje te nove attribute.
 - Nikad ne ulančavaj više naredbi koje kreiraju migracije (npr. `make:model -m`, `make:migration`) pomoću `&&` ili `;` — mogu dobiti identične timestampove. Pokreni svaku naredbu zasebno i pričekaj da završi prije pokretanja sljedeće.
 - **Enumi:** ako PHP Enum postoji za neki domenski koncept, uvijek koristi njegove caseove (ili njihov `->value`) umjesto sirovih stringova — svugdje: u rutama, middlewareu, migracijama, seedovima, konfiguracijama i UI defaultima.
 - **Kontroleri:** kontroleri s jednom metodom trebaju koristiti `__invoke()`; RESTful kontroleri s više metoda trebaju koristiti `Route::resource()->only([])`.
 - Nemoj kreirati kontrolere sa samo jednom metodom koja samo vraća `view()`. Umjesto toga koristi `Route::view()` direktno s Blade datotekom.
 - U Blade predlošcima za prikaz flash poruka uvijek koristi Laravelovu direktivu `@session()` umjesto `@if(session())`.
 - U Blade datotekama uvijek koristi direktive `@selected()` i `@checked()` umjesto HTML atributa `selected` i `checked`. Dobar primjer: `@selected(old('status') === App\Enums\ProjectStatus::Pending->value)`. Loš primjer: `{{ old('status') === App\Enums\ProjectStatus::Pending->value ? 'selected' : '' }}`.
-

Upute za testiranje

Prije pisanja testova

1. **Provjeri shemu baze podataka** — koristi alat `database-schema` da razumiješ: - koji stupci imaju default vrijednosti, - koji stupci mogu biti `null`, - nazive relacija stranih ključeva.
2. **Provjeri nazive relacija** — pročitaj datoteku modela da potvrdiš: - točne nazive metoda relacija (ne pretpostavljaj ih iz naziva stupaca), - povratne tipove i povezane modele.
3. **Testiraj realna stanja** — nemoj pretpostavljati: - da prazan model znači sve `null` vrijednosti (provjeri postoje li defaulti), - da strani ključ `user_id` znači relaciju `user()` (može biti `author()`, `employer()` itd.), - kod testiranja slanja formi koje redirektuju natrag s greškama, provjeri da je stari unos sačuvan pomoću `assertSessionHasOldInput()`.

Filament pravila

- Kad generiraš Filament resurs, **MORAŠ** generirati Filament smoke testove koji provjeravaju radi li resurs. Kad mijenjaš Filament resurs, **MORAŠ** pokrenuti testove (generiraj ih ako ne postoje) i mijenjati resurs/testove dok testovi ne prolaze.
- Kad generiraš Filament resurs, nemoj generirati View stranicu ni Infolist, osim ako to nije izričito traženo.
- Kad referenciraš Filament rute, teži korištenju `getUrl()` umjesto Laravelovog `route()`. Umjesto `route('filament.admin.resources.class-schedules.index')` koristi `ClassScheduleResource::getUrl('index')`. Također, navedi točan naziv resursa umjesto `getResource()`.
- Kad pišeš testove s Pestom, koristi sintaksu `Livewire::test(class)`, a ne `livewire(class)`, kako bi se izbjegla dodatna ovisnost o `pestphp/pest-plugin-livewire`.
- Kad koristiš Enum klasu za polje Eloquent modela, dodaj Enumu sučelja `HasLabel`, `HasColor` i `HasIcon` ako još nisu dodana, umjesto navođenja vrijednosti/labela/boja/ikona unutar Filament formi/tablica. **KRITIČNO:** uvijek koristi točne deklaracije povratnih tipova iz definicija sučelja — nemoj podmetati specifičnije tipove (npr. za `getIcon()` koristi `string|BackedEnum|Htmlable|null`, a ne `string|Heroicon|null`). Kad definiraš default pomoću enuma, nikad ne dodavaj `->value`. Referenca: <https://filamentphp.com/docs/4.x/advanced/enums>
- Uvijek koristi Enum umjesto tvrdo kodirane string vrijednosti gdje je to moguće, ako Enum klasa postoji. Na primjer, u testovima pri kreiranju podataka — ako je polje castano u Enum, koristi taj Enum umjesto tvrdo kodirane vrijednosti.
- Kad dodaješ ikone, uvijek koristi Filamentovu enum klasu `Filament\Support\Icons\Heroicon` umjesto stringa.
- Kad dodaješ akcije koje zahtijevaju autorizaciju, koristi metodu `->authorize('ability')` na akciji umjesto ručnog pozivanja `Gate::authorize()` ili provjere `Gate::allows()`. Metoda `authorize()` automatski rješava i provođenje autorizacije i vidljivost akcije.
- U Filamentu v4 validacijsko pravilo `unique()` po defaultu ima `ignoreRecord: true` — nema potrebe to navoditi.
- U Filamentu v4, ako kreiraš vlastite Blade datoteke s Tailwind klasama, moraš kreirati custom temu i navesti mapu tih Blade datoteka u `theme.css`.
- **Zastarjele v3 metode — NE koristiti:**
 - `->form()` na akcijama/filterima → koristi `->schema()`
 - `->mutateFormUsing()` → koristi `->mutateDataUsing()`

- `Placeholder::make()` → koristi `TextEntry::make()->state()` (import iz `Filament\Infolists\Components\TextEntry`)
- `->label('')` za skrivanje labela → koristi `->hiddenLabel()`